

Conjunctions and disjunctions can also be expressed using logical connectives like `and`, `or`, and `implies`. For instance, the following rule is equivalent to the above two rules combined.

```
rel grandmother(a, c) = (mother(a, b) or father(a, b)) and mother(b, c)
```

Scallop supports value creation by means of foreign functions (FFs). FFs are polymorphic and include arithmetic operators such as `+` and `-`, comparison operators such as `!=` and `>=`, type conversions such as [i32] as `String`, and built-in functions like `$hash` and `$string_concat`. They only operate on primitive values but not relational tuples or atoms. The following example shows how strings are concatenated together using FF, producing the result `full_name("Alice Lee")`.

```
rel first_name("Alice"), last_name("Lee")
rel full_name($string_concat(x, "_", y)) = first_name(x), last_name(y)
```

Note that FFs can fail due to runtime errors such as division-by-zero and integer overflow, in which case the computation for that single fact is omitted. In the example below, when dividing 6 by denominator, the result is not computed for denominator 0 since it causes a FF failure:

```
rel denominator = {0, 1, 2}
rel result(6 / x) = denominator(x) // result contains only integers 3 and 6
```

The purpose of this semantics is to support probabilistic extensions (Section 3.3) rather than silent suppression of runtime errors. When dealing with floating-point numbers, tuples with NaN (not-a-number) are also discarded.

Recursion. A relation r is dependent on s if an atom s appears in the body of a rule with head atom r . A *recursive* relation is one that depends on itself, directly or transitively. The following rule derives additional kinship facts by composing existing kinship facts using the `composition` relation.

```
rel kinship(r3,a,c) = kinship(r1,a,b), kinship(r2,b,c), composition(r1,r2,r3)
```

Negation. Scallop supports stratified negation using the `not` operator on atoms in the rule body. The following example shows a rule defining the `has_no_children` relation as any person p who is neither a father nor a mother. Note that we need to bound p by a positive atom `person` in order for the rule to be well-formed.

```
rel person = {"Alice", "Bob", "Christine"} // can omit () since arity is 1
rel has_no_children(p) = person(p) and not father(_, p) and not mother(_, p)
```

A relation r is *negatively* dependent on s if a negated atom s appears in the body of a rule with head atom r . In the above example, `has_no_children` negatively depends on `father`. A relation cannot be negatively dependent on itself, directly or transitively, as Scallop supports only stratified negation. The following rule is rejected by the compiler, as the negation is not stratified:

```
rel something_is_true() = not something_is_true() // compilation error!
```

Aggregation. Scallop also supports stratified aggregation. The set of built-in aggregators include common ones such as `count`, `sum`, `max`, and first-order quantifiers `forall` and `exists`. Besides the operator, the aggregation specifies the binding variables, the aggregation body to bound those variables, and the result variable(s) to assign the result. The aggregation in the example below reads, “variable n is assigned the count of p , such that p is a person”:

```
rel num_people(n) = n := count(p: person(p))
```

In the rule, p is the binding variable and n is the result variable. Depending on the aggregator, there could be multiple binding variables or multiple result variables. Further, Scallop supports SQL-style group-by using a `where` clause in the aggregation. In the following example, we compute the number of children of each person p , which serves as the group-by variable.

(Tag)	t	$\in T$
(False)	0	$\in T$
(True)	1	$\in T$
(Disjunction)	$\oplus$	$: T \times T \rightarrow T$
(Conjunction)	$\otimes$	$: T \times T \rightarrow T$
(Negation)	$\ominus$	$: T \rightarrow T$
(Saturation)	$\ominus$	$: T \times T \rightarrow \text{Bool}$

Fig. 4. Algebraic interface for provenance.

(Predicate)	p	
(Aggregator)	g	$::= \text{count} \mid \text{sum} \mid \text{max} \mid \text{exists} \mid \dots$
(Expression)	e	$::= p \mid \gamma_g(e) \mid \pi_\alpha(e) \mid \sigma_\beta(e)$ $\mid e_1 \cup e_2 \mid e_1 \times e_2 \mid e_1 - e_2$
(Rule)	r	$::= p \leftarrow e$
(Stratum)	s	$::= \{r_1, \dots, r_n\}$
(Program)	$\bar{s}$	$::= s_1; \dots; s_n$

Fig. 5. Abstract syntax of core fragment of SCLRAM.

```

rel parent(a, b) = father(a, b) or mother(a, b)
rel num_child(p, n) = n := count(c: parent(c, p) where p: person(p))

```

Finally, quantifier aggregators such as `forall` and `exists` return one boolean variable. For instance, in the aggregation below, variable `sat` is assigned the truthfulness (`true` or `false`) of the following statement: “for all `a` and `b`, if `b` is `a`’s father, then `a` is `b`’s son or daughter”.

```

rel integrity_constraint(sat) =
  sat := forall(a, b: father(a, b) implies (son(b, a) or daughter(b, a)))

```

There are a couple of syntactic checks on aggregations. First, similar to negation, aggregation also needs to be stratified—a relation cannot be dependent on itself through an aggregation. Second, the binding variables must be bounded by a positive atom in the body of the aggregation.

3.3 Probabilistic Extensions

Although Scallop is designed primarily for neurosymbolic programming, its syntax also supports probabilistic programming. This is especially useful when debugging Scallop code before integrating it with a neural network. Consider a machine learning programmer who wishes to extract structured relations from a natural language sentence “Bob takes his daughter Alice to the beach”. The programmer could imitate a neural network producing a probability distribution of kinship relations between Alice (A) and Bob (B) as follows:

```

rel kinship = {0.95::(FATHER, A, B); 0.01::(MOTHER, A, B); ... }

```

Here, we list out all possible kinship relations between Alice and Bob. For each of them, we use the syntax `[PROB]::[TUPLE]` to tag the kinship tuples with probabilities. The semicolon “;” separating them specifies that they are mutually exclusive—Bob cannot be both the mother and father of Alice.

Scallop also supports operators to sample from probability distributions. They share the same surface syntax as aggregations, allowing sampling with `group-by`. The following rule deterministically picks the most likely kinship relation between a given pair of people `a` and `b`, which are implicit `group-by` variables in this aggregation. As the end, only one fact, `0.95::top_1_kinship(FATHER, A, B)`, will be derived according to the above probabilities.

```

rel top_1_kinship(r, a, b) = r := top<1>(rp: kinship(rp, a, b))

```

Other types of sampling are also supported, including categorical sampling (`categorical<K>`) and uniform sampling (`uniform<K>`), where a static constant `K` denotes the number of trials.

Finally, rules can also be tagged by probabilities which can reflect their confidence. The following rule states that a grandmother’s daughter is one’s mother with 90% confidence.

```

rel 0.9::mother(a, c) = grandmother(a, b) and daughter(b, c)

```

Probabilistic rules are syntactic sugar. They are implemented by introducing in the rule’s body an auxiliary 0-ary (i.e., boolean) fact that is regarded true with the tagged probability.

(Constant)	$\mathbb{C}$	$\ni$	c	$::=$	$int \mid bool \mid str \mid \dots$	(Tuples)	U	$\in$	$\mathcal{U}$	$\triangleq$	$\mathcal{P}(\mathbb{U})$
(Tuple)	$\mathbb{U}$	$\ni$	u	$::=$	$c \mid (u_1, \dots, u_n)$	(Tagged-Tuples)	U_T	$\in$	$\mathcal{U}_T$	$\triangleq$	$\mathcal{P}(\mathbb{U}_T)$
(Tagged-Tuple)	$\mathbb{U}_T$	$\ni$	u_t	$::=$	$t :: u$	(Facts)	F	$\in$	$\mathcal{F}$	$\triangleq$	$\mathcal{P}(\mathbb{F})$
(Fact)	$\mathbb{F}$	$\ni$	f	$::=$	$p(u)$	(Database)	F_T	$\in$	$\mathcal{F}_T$	$\triangleq$	$\mathcal{P}(\mathbb{F}_T)$
(Tagged-Fact)	$\mathbb{F}_T$	$\ni$	f_t	$::=$	$t :: p(u)$						

Fig. 6. Semantic domains for SCLRAM.

4 REASONING FRAMEWORK

The preceding section presented Scallop’s surface language for use by programmers to express discrete reasoning. However, the language must also support differentiable reasoning to enable end-to-end training. In this section, we formally define the semantics of the language by means of a provenance framework. We show how Scallop uniformly supports different reasoning modes—discrete, probabilistic, and differentiable—simply by defining different provenances.

We start by presenting the basics of our provenance framework (Section 4.1). We then present a low-level representation SCLRAM, its operational semantics, and its interface to the rest of a Scallop application (Sections 4.2-4.4). We next present differentiation and three different provenances for differentiable reasoning (Section 4.5). Lastly, we discuss practical considerations (Section 4.6).

4.1 Provenance Framework

Scallop’s provenance framework enables to tag and propagate additional information alongside relational tuples in the logic program’s execution. The framework is based on the theory of *provenance semirings* [Green et al. 2007]. Fig. 4 defines Scallop’s algebraic interface for provenance. We call the additional information a *tag* t from a *tag space* T . There are two distinguished tags, $\mathbf{0}$ and $\mathbf{1}$, representing unconditionally *false* and *true* tags. Tags are propagated through operations of binary *disjunction* $\oplus$, binary *conjunction* $\otimes$, and unary *negation* $\ominus$ resembling logical *or*, *and*, and *not*. Lastly, a *saturation* check $\ominus$ serves as a customizable stopping mechanism for fixed-point iteration.

All of the above components combined form a 7-tuple $(T, \mathbf{0}, \mathbf{1}, \oplus, \otimes, \ominus, \ominus)$ which we call a *provenance* T . Scallop provides a built-in library of provenances and users can add custom provenances simply by implementing this interface.

Example 4.1. $\max\text{-min-prob}(\text{mmp}) \triangleq ([0, 1], \mathbf{0}, \mathbf{1}, \max, \min, \lambda x. (1 - x), =)$, is a built-in *probabilistic provenance*, where tags are numbers between 0 and 1 that are propagated with $\max$ and $\min$. The tags do not represent true probabilities but are merely an approximation. We discuss richer provenances for more accurate probability calculations later in this section.

A provenance must satisfy a few properties. First, the 5-tuple $(T, \mathbf{0}, \mathbf{1}, \oplus, \otimes)$ should form a semiring. That is, $\mathbf{0}$ is the additive identity and annihilates under multiplication, $\mathbf{1}$ is the multiplicative identity, $\oplus$ and $\otimes$ are associative and commutative, and $\otimes$ distributes over $\oplus$. To guarantee the existence of fixed points (which are discussed in Section 4.3), it must also be *absorptive*, i.e., $t_1 \oplus (t_1 \otimes t_2) = t_1$ [Dannert et al. 2021]. Moreover, we need $\ominus \mathbf{0} = \mathbf{1}$, $\ominus \mathbf{1} = \mathbf{0}$, $\mathbf{0} \oplus \mathbf{1}$, $\mathbf{0} \otimes \mathbf{0}$, and $\mathbf{1} \ominus \mathbf{1}$. A provenance which violates an individual property (e.g. absorptive) is still useful to applications that do not use the affected features (e.g. recursion) or if the user simply wishes to bypass the restrictions.

4.2 SCLRAM Intermediate Language

Scallop programs are compiled to a low-level representation called SCLRAM. Fig. 5 shows the abstract syntax of a core fragment of SCLRAM. Expressions resemble queries in an extended relational algebra. They operate over relational predicates (p) using unary operations for aggregation (γ_g with aggregator g), projection (π_α with mapping α), and selection (σ_β with condition β), and binary operations union ($\cup$), product ($\times$), and difference ($-$).